

Naprawianie automatów skończonych z brakującymi stanami i krawędziami

Repairing Finite Automata with Missing Components

Julia Cygan, Patryk Flama

II UWr

5 lutego 2026

Definicja problemu

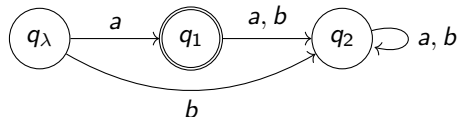
DFA

Deterministyczny automat skończony to krotka:

$$(Q, \Sigma, \delta, q_\lambda, \mathbb{F}_A, \mathbb{F}_R)$$

gdzie:

- Q - zbiór stanów
- Σ - alfabet
- $\delta : Q \times \Sigma \rightarrow Q$ - przejścia
- q_λ - stan początkowy
- $\mathbb{F}_A$ - stany akceptujące
- $\mathbb{F}_R$ - stany odrzucające



Definicja problemu

Z nieokreśloną liczbą stanów

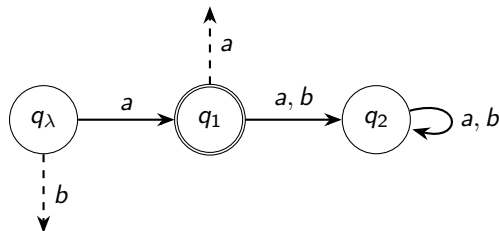
Wejście: Częściowy DFA A z brakującymi krawędziami i stanami, zbiory S^+ , S^- .

Wyjście: Czy można uzupełnić przejścia, klasyfikacje stanów i dodać stany, aby automat był poprawny?

Z określoną liczbą stanów

Wejście: Częściowy DFA A z brakującymi krawędziami, zbiory S^+ , S^- .

Wyjście: Czy można uzupełnić przejścia i klasyfikacje stanów, aby automat był poprawny?
(Bez dodawania nowych stanów)



Dlaczego ten problem jest ciekawy?

Znany trudny problem

Problem *DFA-consistency* (najmniejszy zgodny automat):

- **Wejście:** liczba n , zbiory S^+ , S^-
- **Wyjście:** Czy istnieje DFA z $\leq n$ stanami akceptujący S^+ i odrzucający S^- ?

Przykłady zastosowań

- Problemy uczenia pasywnego, w których mamy częściową wiedzę o rozwiązaniu
- Rekonstrukcja systemu z logów
- Uzupełnianie częściowego oprogramowania

Redukcja problemu

Problem naprawienia DFA z nieznaną liczbą stanów można zredukować do problemu z określoną liczbą stanów.

NP-zupełność

Problem naprawienia częściowego DFA z określoną liczbą stanów jest NP-zupełny.

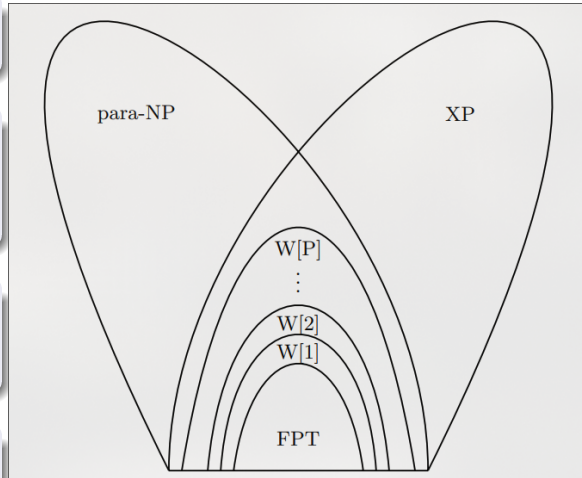
Złożoność parametryczna

Klasa FPT: Problemy rozwiązywalne w czasie $O(f(k) \cdot n^c)$ dla pewnej funkcji f .

Klasa $W[i]$: Problemy z klasy $W[i]$ mogą zostać zredukowane do problemu *weighed circuit satisfiability* dla obwodów o głębokości ograniczonej do i .

Klasa $W[P]$: Problemy rozwiązywalne w czasie $O(n^c)$ przez niedeterministyczną maszynę Turinga używającą $f(k) \cdot \log n$ bitów niedeterminizmu.

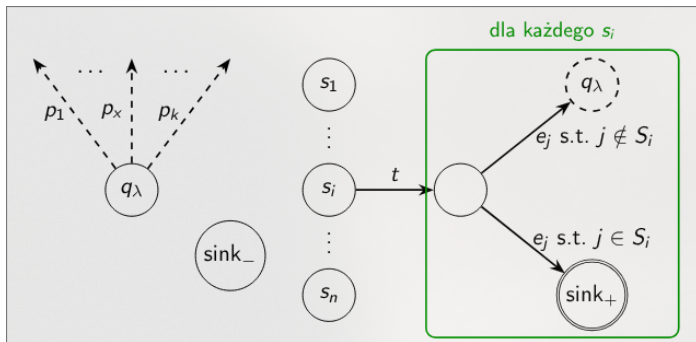
Klasa XP: Problemy rozwiązywalne w czasie $O(n^{f(k)})$ dla pewnej funkcji f .



$W[2]$ -trudność: redukcja z k -set cover

$$\text{dla każdego } j \in \{1, \dots, |U|\} [p_1 \ t \ e_j \ p_2 \ t \ e_j \ \dots \ p_k \ t \ e_j] \in S^+ \quad (1)$$

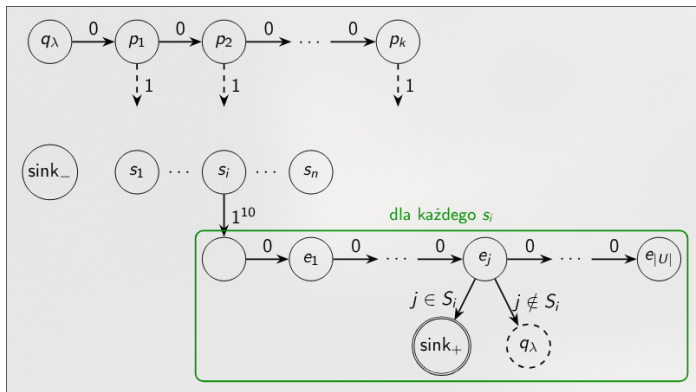
$$\text{dla każdego } x \in \{1, \dots, k\} [p_x] \in S^- \quad (2)$$



$W[2]$ -trudność: stały rozmiar alfabetu

$$\text{dla każdego } j \in \{1, \dots, |U|\} [0^1 1^{11} 0^j 1 0^2 1^{11} 0^j 1 \dots 0^k 1^{11} 0^j 1] \in S^+ \quad (3)$$

$$\text{dla każdego } x \in \{1, \dots, k\} [0^x 1^{11}] \in S^- \quad (4)$$



Niedeterministyczna maszyna Turinga

- 1 **Zgaduj:** $O(k \log |Q|)$ bitów niedeterminizmu
Zagadnij k stanów docelowych brakujących krawędzi
- 2 **Weryfikuj:** Czas wielomianowy
Sprawdź czy automat akceptuje S^+ i odrzuca S^-

Wniosek

Problem należy do $W[P]$ — mamy κ -ograniczoną niedeterministyczną maszynę Turinga.

Rezultat: złożoność naszego problemu

Co to oznacza?

Problem naprawienia k -częściowego DFA jest $W[2]$ -trudny i należy do $W[P]$.

Nasz problem jest nie łatwiejszy niż każdy problem z $W[2]$ i nie trudniejszy niż każdy problem z $W[P]$.

Problem nie jest FPT (chyba że $FPT = W[2]$) - jest trudny parametrycznie

Dlaczego się tym zajmujemy?

Mimo trudności problemu, chcemy znaleźć empirycznie szybszy algorytm.

Brute force

Jako punkt odniesienia bierzemy algorytm naiwny, który dla kolejnych wygenerowanych wariantów automatu symuluje przetwarzanie wszystkich próbek i sprawdza czy każda z nich kończy się w stanie zgodnym z danymi wejściowymi

Algorytm ze skokami (Jump Tables)

Idea

W trakcie przechodzenia po automacie, przeskakujemy ciągi znanych krawędzi (podanych na wejściu).

Idea

Hill climbing z losowymi restartami:

W każdej iteracji rozważamy sąsiednie pojedyncze zmiany: przypisanie innego stanu docelowego do jednego z nieznanych przejść automatu.

Akceptujemy modyfikację, jeśli prowadzi ona do zmniejszenia liczby błędnych próbek.

Po pewnej liczbie iteracji, losujemy nowy punkt w przestrzeni.

Pruning przestrzeni rozwiązań

Idea

Chcielibyśmy jak najszybciej wiedzieć, że dana gałąź przeszukiwań nie ma sensu dalszej eksploracji.

